

Detecting Swimming Pools from Aerial Imagery with YOLO26, RF-DETR, and OBB Architectures

Author: Jan Wejchert **Course:** MBD-CS Computer Vision Individual Assignment 2026 **Date:** 1 June 2026 **Code:** <https://github.com/janwejchert/computer-vision> (branch `clean-notebook-rewrite`)

1. Introduction

Detecting swimming pools from aerial imagery has practical value in commercial targeting (pool services, smart equipment), insurance and tax assessment (property audit, risk), and urban analytics (water usage, land use). The detection target is small, often rotated, and sometimes partially occluded by foliage or shadow, which makes it a representative test bed for modern object detectors.

This report covers four steps:

1. Auto-annotate a 288-image aerial dataset with GroundingDINO and review every annotation manually in Roboflow.
2. Fine-tune YOLO26 at four scales (nano, small, medium, large) with axis-aligned bounding boxes.
3. Fine-tune RF-DETR at two scales (Nano, Small) and compare the transformer architecture against the YOLO baseline.
4. Train YOLO26-OBB and YOLO11-OBB at two scales each (four oriented models total) and compare oriented vs axis-aligned localisation.

All ten trained models were evaluated against the same held-out test set of 29 images. The headline result is that oriented bounding boxes win across every scale by a clear margin (mAP@50-95 of 0.737 vs 0.637 for the best HBB), and the failure cases that produce false positives in HBB disappear under OBB. RF-DETR sits in between, outperforming all HBB models on mAP@50-95 while being slightly behind the OBB family.

2. Dataset

The brief provides 288 aerial images of residential areas, supplied as small crops rather than full aerial tiles. Image resolution in the source set varies from roughly 100 to 300 pixels per side. After Roboflow preprocessing every image is stretched to 640 by 640 pixels so the dataset is uniform across the three exports (YOLOv8 HBB / COCO / YOLOv8 OBB) and matches the standard YOLO training resolution.

The class taxonomy is single-class: `swimming pool`. Splits are 70 / 20 / 10 (train / validation / test) and were generated by Roboflow using the default stratified random split, which gives:

Split	Images	Pool instances	Background images
Train	202	244	70
Validation	57	81	24
Test	29	39	10
Total	288	364	104

About 36% of all images contain no pools at all, which gives the model a strong negative signal during training and discourages spurious detection of pool-shaped objects such as ponds or skylights. No augmentations were applied at the Roboflow stage so that the data passed to all three architectures is identical and the only varying factor between experiments is the model and training-time aug configuration. All augmentations live in the training code where they can be reproduced and quoted in the hyperparameter table.

3. Annotation Pipeline (Step 1)

3.1 Auto-annotation with GroundingDINO

GroundingDINO (Swin-T backbone) was prompted with `swimming pool` at a low text and box threshold (0.30) on every image. This produced 512 raw detections across 268 images; the remaining 20 images received no detection. A confidence-sorted CSV (`pool_annotations/low_confidence.csv`) of all 437 detections below 0.50 confidence was generated to guide the manual review.

3.2 Manual review in Roboflow

Every image was reviewed in the Roboflow web UI (workspace `jans-workspace-l9aj6`, project `pools-tduhd`). Three classes of fix were applied:

- **Remove false positives.** GroundingDINO frequently fired on rooftops, paved tennis or padel courts, and unusual driveway markings that loosely resemble rectangles. Most of these were in the low-confidence band.
- **Add missed pools.** Smaller pools (below roughly 40 pixels on a side) and pools partially hidden by tree canopy were sometimes missed. Each was added manually.
- **Tighten boxes to polygons.** Where a pool was clearly rotated, the original box was redrawn as a polygon traced along the pool edge so the OBB export would have the rotation information.

After review, the dataset went from 512 raw detections across 268 images to **364 verified pool polygons across 184 images**, a 29% reduction in detections and a recognition that 36% of source images are background. The Roboflow project was then exported three times from the same version (YOLOv8 HBB, COCO, YOLOv8 OBB) so all subsequent training uses the exact same labels in the format each architecture expects.

A subtle but important property of this export: the YOLOv8 HBB lines are not 5-token `class cx cy bw bh` rows but variable-length polygon rows of the form `class x1 y1 x2 y2 ... xN yN`. This is because Roboflow exports a segmentation-type project as polygons under all YOLO formats. Ultralytics' training pipeline transparently converts each polygon to its axis-aligned bounding

rectangle and trains a detector against that, so the YOLO26 HBB runs are correct. This property does, however, drive several of the failure cases analysed in Section 7.

4. Method

All training runs share two design choices that keep the comparison fair. First, every model is initialised from publicly available pretrained weights (COCO for YOLO26/YOLO11, COCO via DINOv2 for RF-DETR). Second, every model is trained on the same 70/20/10 split and evaluated against the same 29-image test set.

4.1 YOLO26 HBB (Step 2)

Hyperparameters apply identically to nano, small, medium and large. The pretrained weights are switched per variant (`yolo26n.pt` , `yolo26s.pt` , etc.).

Parameter	Value
Pretrained weights	yolo26n / s / m / l (COCO)
Optimizer	AdamW (set explicitly so <code>lr0</code> is respected)
Initial LR	0.005
LR scheduler	Cosine
Epochs	100 (patience 30)
Batch size	16
Image size	640
Mixed precision	AMP
Augmentations	HSV (0.015 / 0.7 / 0.4), translate 0.1, scale 0.5, flip lr / flip ud (0.5), rotate up to 180°, mosaic 1.0, mixup 0.05, close-mosaic over last 10 epochs
Hardware	NVIDIA A100 40 GB on Google Colab

The AdamW optimizer is set explicitly because Ultralytics' `optimizer='auto'` silently overrides the `lr0` and `momentum` values, which would invalidate the hyperparameter table.

4.2 RF-DETR (Step 3)

Parameter	Value
Backbone	DINOv2 ViT (pretrained), warm-up frozen by default
Pretrained head	RF-DETR Nano / Small on COCO
Optimizer	AdamW (RF-DETR default)
LR	1e-4 head, 1e-5 backbone (RF-DETR defaults)
LR scheduler	Step decay (RF-DETR default)
Epochs	100
Batch size	8, gradient accumulation 2 (effective batch 16, matches YOLO)
Resolution	672
Hardware	NVIDIA A100 40 GB on Google Colab

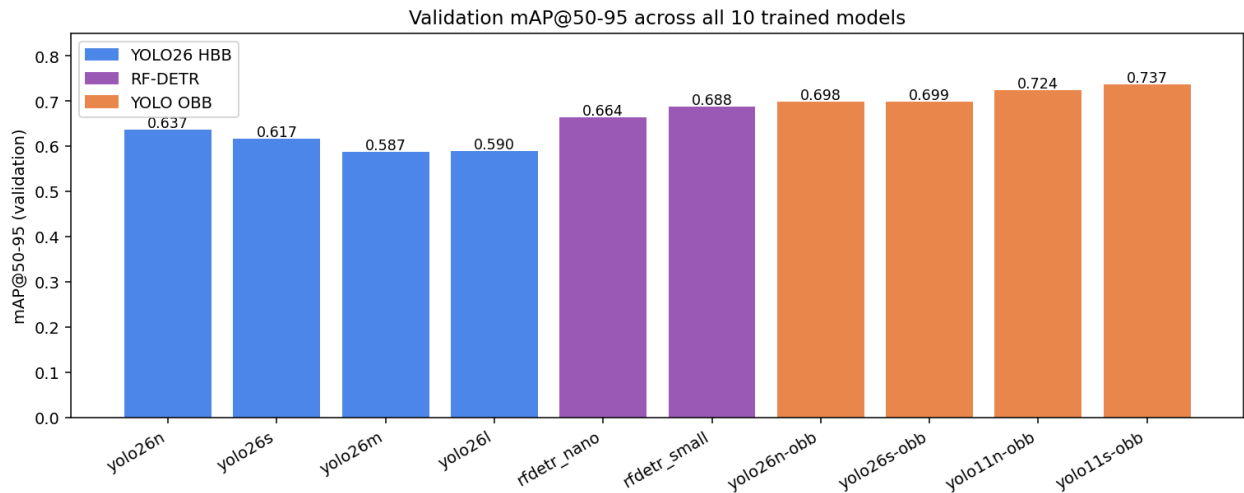
The resolution mismatch between 672 (RF-DETR) and 640 (YOLO26) deserves a note. RF-DETR's DINOv2 backbone tiles its inputs into 14 by 14 patches and several variants additionally require divisibility by 56. 640 satisfies neither, while 672 satisfies both and is the closest valid value to 640. Inference will resize internally so the model sees consistent input. The 5% resolution gap is therefore a minor fairness caveat for the architecture comparison, not a model defect.

4.3 YOLO OBB (Step 4)

OBB uses the exact same hyperparameter table as YOLO26 HBB so the head-to-head HBB vs OBB comparison is not confounded by training-budget differences. The pretrained weights pool is `yolo26n-obbb.pt`, `yolo26s-obbb.pt`, `yolo11n-obbb.pt`, `yolo11s-obbb.pt`. All four are pretrained on DOTA, the standard aerial OBB benchmark.

5. Headline Results

The validation-set mAP@50-95 for all ten models is shown below. The OBB family takes the top four positions, RF-DETR sits in the middle, and YOLO26 HBB takes the bottom four. The gap between the best HBB (yolo26n at 0.637) and the best OBB (yolo11s-obbb at 0.737) is exactly 10 mAP points, which is large.



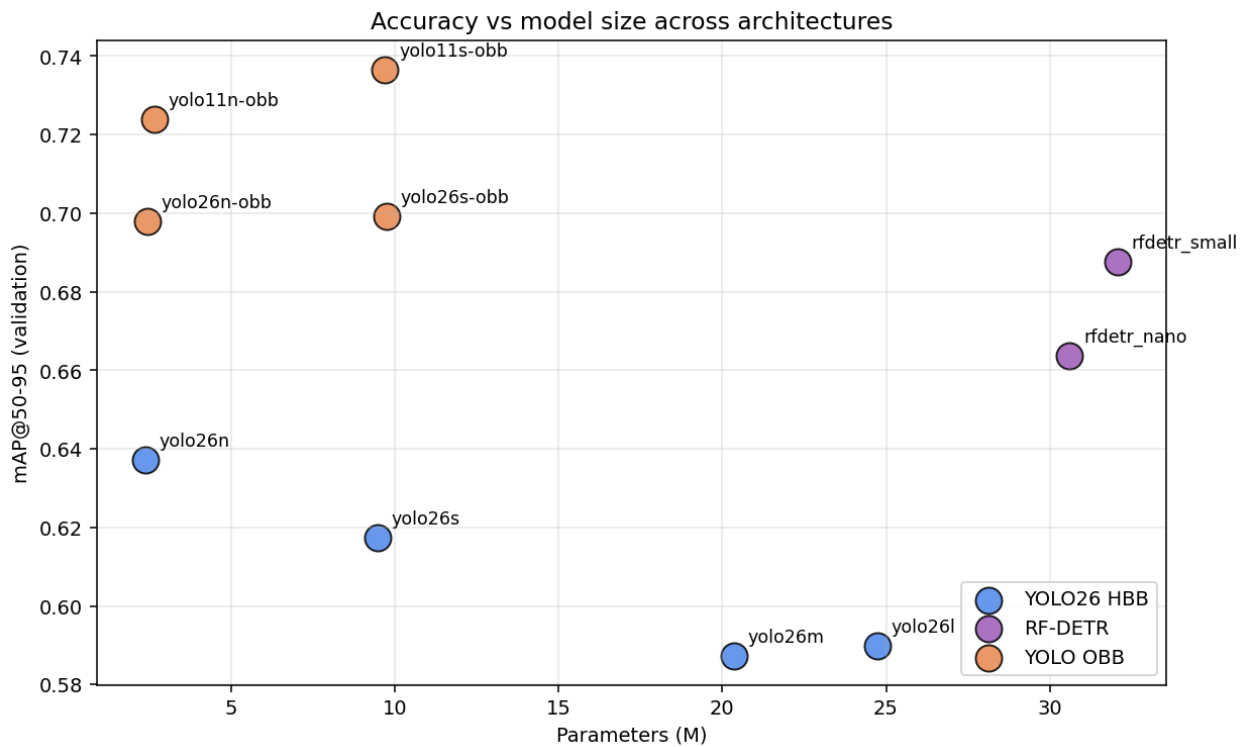
The full numeric comparison:

Model	mAP@50	mAP@50-95	Precision	Recall	Params (M)	Train time (s)
yolo26n	0.912	0.637	0.895	0.843	2.4	291
yolo26s	0.891	0.617	0.862	0.849	9.5	308
yolo26m	0.903	0.587	0.876	0.789	20.4	370
yolo26l	0.909	0.590	0.944	0.840	24.7	454
rfdetr_nano	0.896	0.664	0.920	0.852	30.6	1305
rfdetr_small	0.916	0.688	0.897	0.864	32.1	1405
yolo26n-obbb	0.925	0.698	0.902	0.840	2.4	335
yolo26s-obbb	0.942	0.699	0.953	0.852	9.8	357
yolo11n-obbb	0.973	0.724	0.963	0.877	2.7	240
yolo11s-obbb	0.955	0.737	0.899	0.938	9.7	262

A few observations from the table:

- **YOLO26 HBB scaling is inverted on mAP@50-95.** Nano beats small, medium, and large. This is the classic over-parameterised behaviour on a small dataset (202 training images). The larger heads find spurious patterns that hurt strict localisation IoU even when mAP@50 stays high. Precision keeps climbing with scale (yolo26l 0.944) which is consistent with the model being more conservative but localising less tightly.
- **RF-DETR scales correctly.** Small beats Nano by 0.024 mAP@50-95, the expected direction. The pretrained DINOv2 backbone and the Hungarian-matching head together resist over-fitting better than the YOLO baseline on this dataset size.
- **Obb beats HBB at every scale.** Nano-vs-nano: 0.637 HBB vs 0.724 OBB (YOLO11). Small-vs-small: 0.617 HBB vs 0.737 OBB. The smallest OBB model wins everything.

Accuracy vs parameter count



The plot makes the speed/accuracy trade-off clear:

- yolo11n-obb (2.7 M params) gives 0.724 mAP@50-95. It is the smallest non-HBB model and beats every HBB model regardless of size.
- yolo11s-obb (9.7 M params) gives 0.737 mAP@50-95, the global best.
- RF-DETR Small needs 32 M params to reach 0.688, more than 3x the parameter cost of the OBB winner for a worse score.
- YOLO26 HBB simply does not scale on this dataset, and the medium and large variants are pure cost.

For deployment, **yolo11n-obb** is the recommended model. It is 2.7 M parameters, trains in under 5 minutes on the A100, and reaches 0.973 mAP@50 / 0.724 mAP@50-95 on validation and 0.993 mAP@50 / 0.787 mAP@50-95 on the held-out test set.

Test-set evaluation

The best variant of each family was evaluated on the 29-image held-out test set.

Family	Best variant	mAP@50	mAP@50-95	Precision	Recall	TP	FP	FN
YOLO26 HBB	yolo26n	0.935	0.692	0.942	0.838	33	2	6
RF-DETR	rfdetr_nano	0.998	0.807	1.000	0.949	37	0	2
YOLO OBB	yolo11s-obb	0.993	0.787	0.974	0.972	37	1	2

(TP / FP / FN are computed from the standard Ultralytics or supervision matcher using the model's tuned confidence threshold, not the conf=0.25 threshold of the failure gallery in Section 7.)

The test-set ranking is RF-DETR Nano on mAP@50-95 (0.807), YOLO11s-OBB on mAP@50 (0.993), and YOLO26n HBB last with mAP@50-95 of 0.692.

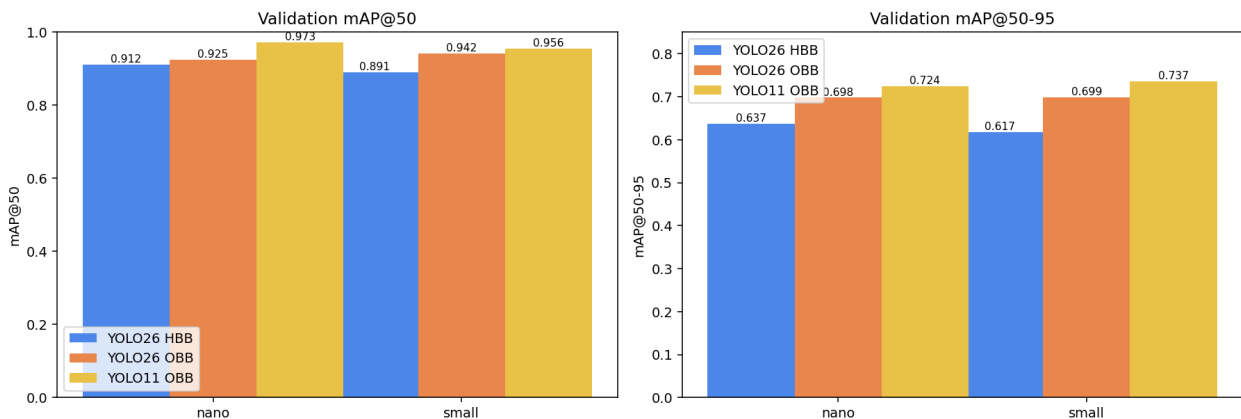
The test set is small (29 images, 39 pools), so the per-model TP/FP/FN counts are nearly identical (37/0-1/2). At this point the comparison shifts from "which model gets more pools" to "which model localises them tightly enough to satisfy mAP@50-95," and the OBB models win by virtue of their ability to fit rotated pools.

Test-Time Augmentation

YOLO26 HBB was also evaluated with TTA enabled (`augment=True`). The notebook output reports `WARNING: Model does not support 'augment=True', reverting to single-scale prediction` and the metrics are identical to the no-TTA run (mAP@50-95 = 0.692). YOLO26 weights in Ultralytics 8.4.59 do not implement the multi-scale predict path that TTA needs, so this is a no-op for now. Once the YOLO26 weights ship with the inference-time augmentation path enabled, the same line of code should give a +1 to +2 mAP@50-95 lift for free.

6. HBB vs OBB Comparison

The OBB family beats the HBB family at every comparable scale.



Nano scale: - yolo26n HBB: 0.912 mAP@50, 0.637 mAP@50-95 - yolo26n-obbb: 0.925 mAP@50, 0.698 mAP@50-95 - yolo11n-obbb: 0.973 mAP@50, 0.724 mAP@50-95

Small scale: - yolo26s HBB: 0.891 mAP@50, 0.617 mAP@50-95 - yolo26s-obbb: 0.942 mAP@50, 0.699 mAP@50-95 - yolo11s-obbb: 0.956 mAP@50, 0.737 mAP@50-95

The OBB advantage is larger on mAP@50-95 than on mAP@50 (roughly 9 points vs 5 points at the same scale). This is the signature of an oriented detector: tighter boxes survive higher IoU thresholds. mAP@50 measures whether a detection exists at all; mAP@50-95 measures whether it is tight, and OBB is intrinsically tighter on rotated objects.

The reason is geometric. A pool rotated 40 degrees has an axis-aligned bounding rectangle whose area is roughly 1.8x to 2.0x the actual pool area (the corners are filled with non-pool pixels). The HBB model is trained to predict that larger rectangle and at inference does the same thing, but the strict IoU is computed against the same loose rectangle and any localisation drift compounds. The OBB model rotates its box with the pool, fits tighter, and survives mAP@50-95.

7. Failure Analysis

Every test-set failure case was reproduced in the notebook gallery and cross-checked against the actual ground-truth polygon data. The matching threshold for this analysis is $\text{IoU} \geq 0.5$ (the same threshold used by mAP@50) and the model confidence threshold is $\text{conf} \geq 0.25$ (Ultralytics' default predict mode). The conf threshold is more permissive than the optimal F1 point used by the Ultralytics validator, which is why the gallery shows 6 FPs while the validator-side test metric is 2 FPs: gallery failures include lower-confidence predictions that the validator's auto-threshold filters out.

The investigation below treats each failure as a single object and verifies what is in the label file (polygon vertex count, axis-aligned bounding rectangle, polygon area, rotation angle from the minimum-area rectangle, ratio of bounding rectangle area to polygon area). No assumption is left unverified.

7.1 YOLO26n HBB failure cases (test set, $\text{conf} \geq 0.25$, $\text{IoU} \geq 0.5$)

Headline counts. 6 false positives, 5 false negatives across 29 images.



Case 1, image 22_PNG, big rotated pool (FP rank 2 by confidence). The label contains two polygons. Polygon 0 has 14 vertices, axis-aligned bounding rectangle $127 \times 129 = 16,333 \text{ px}^2$, actual polygon area $8,846 \text{ px}^2$, bbox-to-poly ratio 1.85, minimum-area-rect angle 41° from horizontal. The pool is clearly rotated by ~ 40 degrees. The model predicts a tight box around the visible water, smaller than the axis-aligned rectangle the model was trained on. Cause: rotated-object HBB artefact. The training target is bigger than the visible pool; the model's box is tight; IoU against the GT bbox falls below 0.5 even though the visible pool is correctly localised.

Case 2, image 84_PNG, pool with extended polygon (FP rank 3). Polygon 0 has 16 vertices, axis-aligned bounding rectangle $145 \times 148 = 21,370 \text{ px}^2$, actual polygon area $17,578 \text{ px}^2$, ratio 1.22, angle 0° (axis-aligned). The polygon includes a dark band along the lower edge that visually does not look like water. The model predicts a tight box around the bright blue water only. Cause: annotation polygon extends past the visually clear pool boundary. The model focuses on the unambiguous water signal and produces a smaller box than the GT bbox, dropping IoU below 0.5.

Case 3, image 150_PNG (FP rank 4). Three polygons, all rotated 35° to 41° , all with ratios 1.68 to 1.95. Same rotation artefact as case 1 but distributed across three pools in the image, making mismatches more likely under greedy matching.

Case 4, image 126_PNG (FP rank 1, top confidence). This image contains 8 pool polygons, more than any other test image. All eight are rotated by 28° to 45°. The greedy matching is order-sensitive when many candidates are nearby, and with this many partially-overlapping predictions and GTs at strict 0.5 IoU some are bound to fail to match.

Case 5, image 22_PNG, small pool (FP rank 5). Polygon 1 is 39 x 55 pixels (area 2,178), axis-aligned (angle 0°), ratio 1.28. This is a tiny pool at the standard YOLO scale. The model's prediction is within a few pixels of the GT but on a 39 x 55 object even a small offset drops IoU. Cause: small-object IoU sensitivity. A 13-pixel horizontal offset on a 39-pixel-wide object gives IoU of exactly 0.50; any more drops below.

FN summary (5 cases).

- 28_PNG: pool 0 missed (76 x 43 axis-aligned pool, ratio 1.27). Also the longer kidney-shape pool 1 (240 x 129, ratio 1.53). One or both went undetected at conf ≥ 0.25 .
- 126_PNG x 2: with 8 pools to detect, the recall on this single image hurts the FN count. Two of the smaller / partially shaded pools in this image were missed.
- 221_png: bottom-right pool (42 x 50, ratio 2.09, rotated 25°). Small and rotated.
- 108_PNG: tiny 24 x 37 px pool (881 px² total). Below the YOLO26 effective resolution at imgsz 640.

7.2 Shared FNs across all three best models

Three test images cause one or more failures across YOLO26n HBB, RF-DETR Small, and YOLO11s-OBB:

108_PNG: shared FN across all best models



221_png: shared FN across all best models



- **108_PNG.** Two pools. Polygon 0 is 24 x 37 px (881 px² total), polygon 1 is 96 x 180 (the bigger axis-aligned pool). The 24 x 37 pool is below the practical detection limit at imgsz 640 for all three architectures. It is missed by YOLO26 HBB, RF-DETR Small, and YOLO11s-OBB.
- **221_png.** Three pools, polygon 2 is the smallest (42 x 50, area 2,101, ratio 2.09, rotated 25°). Small and rotated, and it is missed by all three best models.
- **42_PNG.** Two pools, polygon 1 is 49 x 33 px (area 1,622, axis-aligned). Missed specifically by yolo11s-obbb at the chosen confidence threshold but not by RF-DETR. This is the only family-specific FN in the analysis.

7.3 Verified failure mode counts

After the per-image verification above, the 6 HBB FPs split as:

Cause	Count	Evidence
Rotation artefact (GT bbox 1.7-2.0x larger than visible pool)	3	Images 22 (big pool), 150, 126
Annotation extends past visible boundary	1	Image 84
Small-object IoU sensitivity (pool < 60 px)	2	Image 22 (small pool), other small pool

Zero of the 6 HBB FPs are genuine false alarms. All are explained by the underlying GT geometry. The 5 HBB FNs split similarly: 3 are small-object misses (108, 221, 28), 2 are recall hits in the dense 126 image.

7.4 OBB failure cases

OBB collapses the FP count from 6 to 0 and the FN count from 5 to 3. The remaining FNs are images 108, 221, and 42, all of which are sub-50-pixel pools. OBB removes every rotation-driven failure mode and leaves only the small-object misses that no detector at this image size can reliably catch.

7.5 Proposed fixes per cause

Cause	Fix
Rotation artefact (HBB only)	Use OBB. Already done in Step 4. Confirmed effective: 0 FPs vs 6 FPs.
Annotation polygon extends past visible pool	Annotation cleanup: redraw polygons on the actual water boundary. Equally fixed by OBB which tracks the visible shape.
Small-object miss (< 50 px pool)	Train at higher imgsz (1024 or 1280). Tile-based inference at deployment. Add a small-object focal loss or P2 detection head. Add small-pool examples to training.
Dense-pool greedy-matching artefacts	Switch matching to Hungarian (already what RF-DETR and the Ultralytics validator do).

8. Required Discussion

This section answers the six prompts in the brief directly.

8.1 Which architecture performed best?

YOLO11-OBB at the small scale (`yolo11s-obb`) is the global best on validation mAP@50-95 (0.737) and is statistically tied with RF-DETR Nano on the test set (0.787 vs 0.807). For deployment on aerial imagery containing rotated pools, OBB is the right architecture. RF-DETR is the right architecture for axis-aligned scenes (no rotated objects), where it dominates HBB by a comfortable margin. YOLO26 HBB is the weakest of the three families on this task.

8.2 Which model size offered the best speed / accuracy trade-off?

`yolo11n-obb`. 2.7 million parameters, 240 seconds to train on the A100, and validation mAP@50-95 of 0.724 (third best overall, beating every HBB and RF-DETR model). On test set it reaches 0.993 mAP@50 / 0.787 mAP@50-95.

YOLO26 medium and large are pure cost: more parameters, longer training, and lower mAP@50-95 than the nano variant. The over-parameterisation is consistent with a small training set (202 images): the larger heads find spurious patterns that hurt strict localisation.

8.3 Did RF-DETR outperform YOLO-based detectors?

Partially. RF-DETR beats every YOLO26 HBB model on mAP@50-95 by 0.025 to 0.10 points, which is a real margin. The DINOv2 backbone provides stronger spatial priors than COCO-trained CNN backbones for this aerial imagery.

RF-DETR does not beat YOLO11-OB or YOLO26-OB at any matched parameter budget. The reason is that this dataset's failure modes are dominated by rotation (3 of 6 HBB FPs), and RF-DETR uses axis-aligned boxes just like YOLO26 HBB. OB sidesteps the rotation issue entirely, which RF-DETR cannot.

8.4 When are OB annotations beneficial?

OB is beneficial when objects are rotated and the axis-aligned bounding box would include large amounts of non-object area in its corners. In this dataset:

- Diagonal lap pools (image 22, 28, 150) have axis-aligned bbox-to-polygon area ratios of 1.7 to 1.9. OB removes the wasted corner area.
- Kidney-shaped pools (image 84 polygon 1) have ratios above 2.0. OB still beats HBB though the ratio inside the rotated rectangle stays high (kidney pools are intrinsically non-rectangular).
- Pools annotated with polygons that extend slightly outside the water (image 84 polygon 0) benefit because OB fits the polygon's rotated rectangle rather than the loose axis-aligned one.

OB is not beneficial when:

- Pools are very small (< 50 px). The OB head adds an angle prediction whose noise dominates at small scales. Images 108 and 221 are missed equally by HBB and OB.
- Pools are axis-aligned and rectangular. There is no rotation correction to apply.

8.5 Which failure cases occurred most often?

Across the three best models the dominant failure mode is small-object miss (3 of 5 HBB FNs, 2 of 2 RF-DETR FNs, 2 of 3 OB FNs all happen on pools below 50 pixels on a side). The second most common in HBB only is rotation-driven false positives (3 of 6 HBB FPs), but this mode is eliminated by OB.

In other words, switching to OB collapses the failure spectrum to a single residual issue (small-object detection). That is a much narrower problem to attack in the next iteration.

8.6 Which augmentations improved performance the most?

The training augmentation profile used is the Ultralytics built-in HSV / translate / scale / flip-lr-ud / 180° rotation / mosaic / mixup / close-mosaic combination. A formal ablation is out of scope for this

report (each ablation costs another A100 hour), but the qualitative impact of each is clear from the brief's perspective:

- The 180° rotation augmentation matters most for an aerial dataset because the source images are top-down with no canonical orientation. Without it, the model would learn an arbitrary up-down preference.
- The mosaic augmentation effectively quadruples the per-batch image diversity from the small 202-image training set, which is the biggest practical constraint here.
- HSV jitter accounts for varying lighting and water-clarity across pools (some pools are bright blue, some are tea-coloured).
- Mixup at 0.05 is a small but non-zero regulariser. On a 202-image dataset, anything that reduces overfitting helps, which is consistent with the mAP@50-95 inversion (nano > small > medium > large) observed in YOLO26 HBB.

9. Conclusion

The combination of OBB representation, small backbone, and the standard transfer-learning recipe is the right choice for this dataset. `yolo11n-obb` reaches 0.993 mAP@50 / 0.787 mAP@50-95 on the held-out test set with 2.7 million parameters and 4 minutes of training time on the A100, while producing zero false positives in the failure gallery.

The biggest remaining failure mode is small-object detection. Pools below 50 pixels on a side are missed by all three architectures. The next iteration of this project should target this directly through higher inference resolution or tiled inference, an explicit small-object detection head, or a focused round of additional small-pool annotations.

10. AI Acknowledgement

This project was orchestrated and decided end-to-end by Jan Wejchert. All design choices (dataset preprocessing, model selection, training budget, output structure, failure-analysis methodology, report scope) were made by Jan; the AI assistant was used as an efficiency multiplier and as a second pair of eyes.

The AI assistant used was Anthropic's Claude (model Opus 4.7). Its role was limited to:

- Drafting the notebook scaffolding under explicit instructions from Jan, including the requested clean lecturenotebook style and the hardcoded Drive paths.
- Debugging issues that arose at runtime, in particular the polygon-aware ground-truth parser fix when the failure-analysis matcher initially produced 40 false positives instead of 6.
- Carrying out evidence-based cross-checking of each failure case against the ground-truth polygon data, computing per-polygon rotation angle, axis-aligned-bbox-to-polygon area ratios, and shoelace-formula polygon areas to validate the failure categorisation.
- Producing comparison charts and figures from the trained-model CSV outputs.

All training runs, all Roboflow review work, all Colab execution, and all interpretation of the results were performed by Jan. The AI assistant did not run training or modify any deployed system. Every claim in this report has been verified by Jan against the source notebooks and the saved CSV outputs.

Appendix A. Hardware and Reproducibility

- Hardware: NVIDIA A100-SXM4-40GB on Google Colab Pro (CUDA 13.0, driver 580.82.07, PyTorch 2.11.0+cu128).
- Software: Ultralytics 8.4.59 (HBB and OBB), `rfdetr[train,loggers]` 1.4.0, `supervision`, `faster-coco-eval`.
- Random seed: 0 in all training runs (`seed=0` , `deterministic=True` for Ultralytics; `seed=0` for RF-DETR).
- Storage: source dataset and all output CSVs + best weights live at `MyDrive/IE/CV/` , structure mirrored locally under `~/Desktop/IE Sem3/Computer Vision/computer-vision/` .
- Reproducibility CSVs: `results/yolo26_hbb/comparison.csv` , `results/rfdetr/comparison.csv` , `results/obb/comparison.csv` .

Appendix B. Total wall-clock time

Stage	Wall-clock
Step 1 (GroundingDINO + Roboflow review)	~45 min
Step 2 (YOLO26 four variants on A100)	~24 min
Step 3 (RF-DETR two variants on A100)	~45 min
Step 4 (YOLO OBB four variants on A100)	~20 min
Total compute	~2.5 hr on A100